Smart Log Analyzer & Intrusion Detection Dashboard

1) Building the Real Attack Detector from Logs

Create Your First Log File (Fake Security Logs)

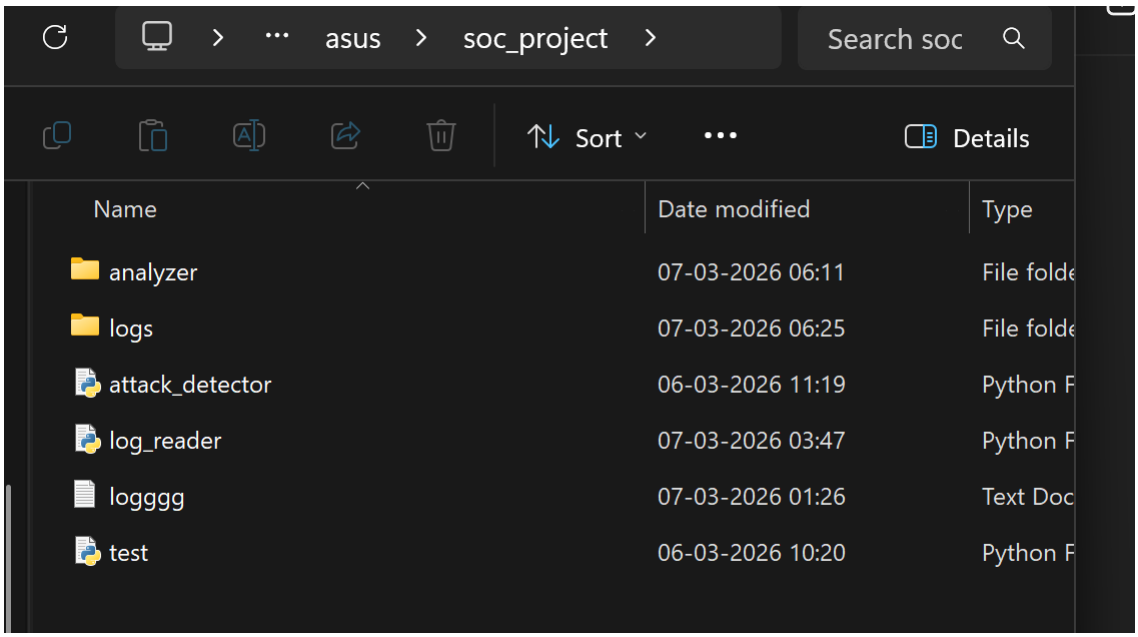
Create a file called: logs.txt

Step 1 - Go to Your Project Folder

Open the folder: soc_project

Your folder should look like this:

soc_project
|— logs
|— analyzer
|— dashboard
| — README.md



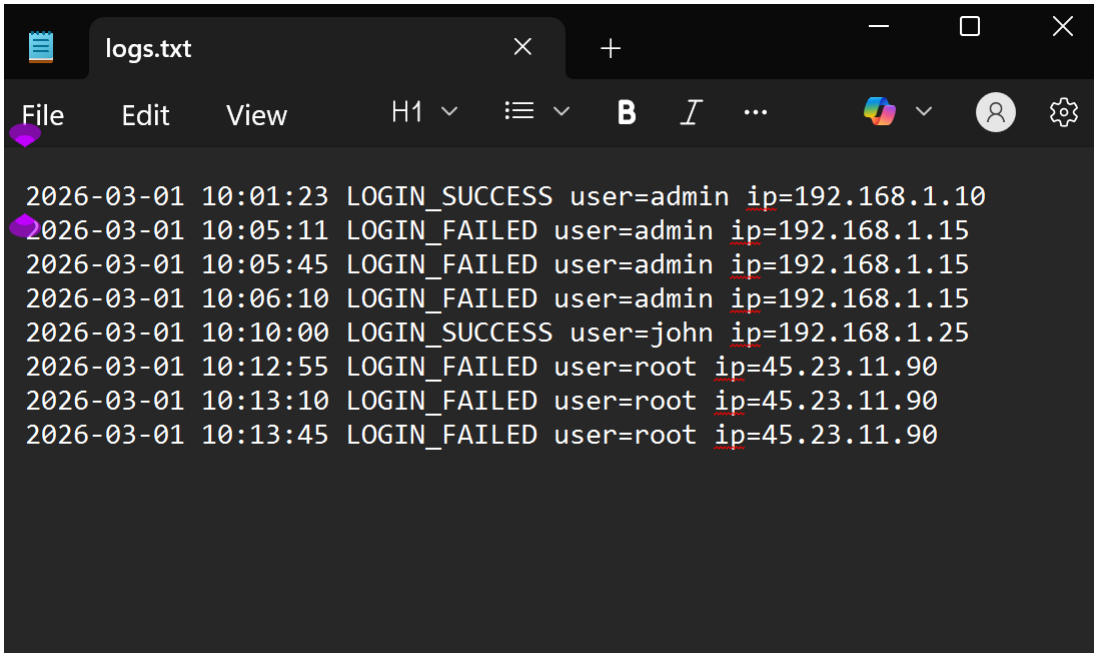
Step 2 - Create Log File

Inside **logs folder**

Create new file: logs.txt

Path: soc_project/logs/logs.txt

Data inside the logs.txt is below:



What This Log Means:

Example line:

2026-03-01 10:05:11 LOGIN_FAILED user=admin ip=192.168.1.15

Part	Meaning
2026-03-01	Date
10:05:11	Time
LOGIN_FAILED	Event
user=admin	Username
ip=192.168.1.15	Attacker IP

Step 4 - Create Analyzer Script

Go to: soc_project/analyzer

Create file: log_analyzer.py

Step 5 - Write This Code

Write below code:

log_analyzer.py - C:\Users\asus\soc_project\analyzer\log_analyzer.py (3.14.3)

File Edit Format Run Options Window Help

```
log_file = "C:/Users/asus/soc_project/logs/logs.txt"

with open(log_file, "r") as file:
    logs = file.readlines()

print("Security Logs:\n")

for log in logs:
    print(log.strip())
```

Step 6 - Run the Script

Run: python log_analyzer.py

Output:

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========
Traceback (most recent call last):
  File "C:\Users\asus\soc_project\analyzer\log_analyzer.py", line 3, in <module>
    with open(log_file, "r") as file:
FileNotFoundError: [Errno 2] No such file or directory: 'C:/Users/asus/soc_proje
ct/logs/logs.txt'
>>>
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========
Security Logs:

2026-03-01 10:01:23 LOGIN_SUCCESS user=admin ip=192.168.1.10
2026-03-01 10:05:11 LOGIN_FAILED user=admin ip=192.168.1.15
2026-03-01 10:05:45 LOGIN_FAILED user=admin ip=192.168.1.15
2026-03-01 10:06:10 LOGIN_FAILED user=admin ip=192.168.1.15
2026-03-01 10:10:00 LOGIN_SUCCESS user=john ip=192.168.1.25
2026-03-01 10:12:55 LOGIN_FAILED user=root ip=45.23.11.90
2026-03-01 10:13:10 LOGIN_FAILED user=root ip=45.23.11.90
2026-03-01 10:13:45 LOGIN_FAILED user=root ip=45.23.11.90
>>>
```

1) Detect Brute Force Attacks

Step 1 - Open Your Script

Open: soc_project/analyzer/log_analyzer.py

Code: New Code

log_analyzer.py - C:\Users\asus\soc_project\analyzer\log_analyzer.py (3.14.3)

File Edit Format Run Options Window Help

```python
log_file = "C:/Users/asus/soc_project/logs/logs.txt"

failed_ips = {}

with open(log_file, "r") as file:
    logs = file.readlines()

for log in logs:

    if "LOGIN_FAILED" in log:

        parts = log.split("ip=")
        ip = parts[1].strip()

        if ip in failed_ips:
            failed_ips[ip] += 1
        else:
            failed_ips[ip] = 1

print("\nLogin Failure Count:\n")

for ip, count in failed_ips.items():
    print(ip, "=", count)

    if count >= 3:
        print("⚠ ALERT: Possible Brute Force Attack from IP", ip)
```

Output:

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py ========

Login Failure Count:

192.168.1.15 = 3
⚠ ALERT: Possible Brute Force Attack from IP 192.168.1.15
45.23.11.90 = 3
⚠ ALERT: Possible Brute Force Attack from IP 45.23.11.90
>>>
```

2) Create a Security Report

Step 1 - Open Your File

Open: soc_project/analyzer/log_analyzer.py

Step 2 - Write Different Code

log_analyzer.py - C:\Users\asus\soc_project\analyzer\log_analyzer.py (3.14.3)

File Edit Format Run Options Window Help

```python
log_file = "C:/Users/asus/soc_project/logs/logs.txt"

failed_ips = {}

total_logs = 0
failed_logins = 0
success_logins = 0
brute_force_count = 0

with open(log_file, "r") as file:
    logs = file.readlines()

for log in logs:

    total_logs += 1

    if "LOGIN_SUCCESS" in log:
        success_logins += 1

    if "LOGIN_FAILED" in log:
        failed_logins += 1

        parts = log.split("ip=")
        ip = parts[1].strip()

        if ip in failed_ips:
            failed_ips[ip] += 1
        else:
            failed_ips[ip] = 1


print("\n===== SECURITY REPORT =====\n")

print("Total Logs:", total_logs)
print("Successful Logins:", success_logins)
print("Failed Logins:", failed_logins)

print("\nBrute Force Detection:\n")

for ip, count in failed_ips.items():

    if count >= 3:
        brute_force_count += 1
        print("⚠ ALERT: Brute Force Attack from IP", ip)

print("\nTotal Brute Force Attacks:", brute_force_count)
```

Run Program

Run: python log_analyzer.py

Output:

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========

===== SECURITY REPORT =====

Total Logs: 15
Successful Logins: 5
Failed Logins: 9

Brute Force Detection:

⚠ ALERT: Brute Force Attack from IP 192.168.1.15
⚠ ALERT: Brute Force Attack from IP 45.23.11.90
⚠ ALERT: Brute Force Attack from IP 192.168.1.30

Total Brute Force Attacks: 3
>>>
```

3) Export Security Data (for Dashboard)

Step 1 - Open Your Script

Open: soc_project/analyzer/log_analyzer.py

Step 2 - Replace Your Code

log_analyzer.py - C:\Users\asus\soc_project\analyzer\log_analyzer.py (3.14.3)

File Edit Format Run Options Window Help

```python
log_file = "C:/Users/asus/soc_project/logs/logs.txt"

report_file = "C:/Users/asus/soc_project/logs/security_report.txt"

failed_ips = {}

total_logs = 0
failed_logins = 0
success_logins = 0
brute_force_count = 0

with open(log_file, "r") as file:
    logs = file.readlines()

for log in logs:

    total_logs += 1

    if "LOGIN_SUCCESS" in log:
        success_logins += 1

    if "LOGIN_FAILED" in log:
        failed_logins += 1

        parts = log.split("ip=")
        ip = parts[1].strip()

        if ip in failed_ips:
            failed_ips[ip] += 1
        else:
            failed_ips[ip] = 1

for ip, count in failed_ips.items():
    if count >= 3:
        brute_force_count += 1


report = f"""
===== SECURITY REPORT =====

Total Logs: {total_logs}
Successful Logins: {success_logins}
Failed Logins: {failed_logins}
Brute Force Attacks: {brute_force_count}
"""

print(report)

with open(report_file, "w") as file:
    file.write(report)

print("Report saved to security_report.txt")
```

Output:

```
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========

===== SECURITY REPORT =====

Total Logs: 15
Successful Logins: 5
Failed Logins: 9
Brute Force Attacks: 3

Report saved to security_report.txt
```

4) Create Your First Cybersecurity Dashboard

Step 1 - Install Chart Library

Open terminal.

Run: pip install matplotlib

Step 2 - Create Dashboard Script

Go to folder: soc_project/dashboard

Create file: dashboard.py

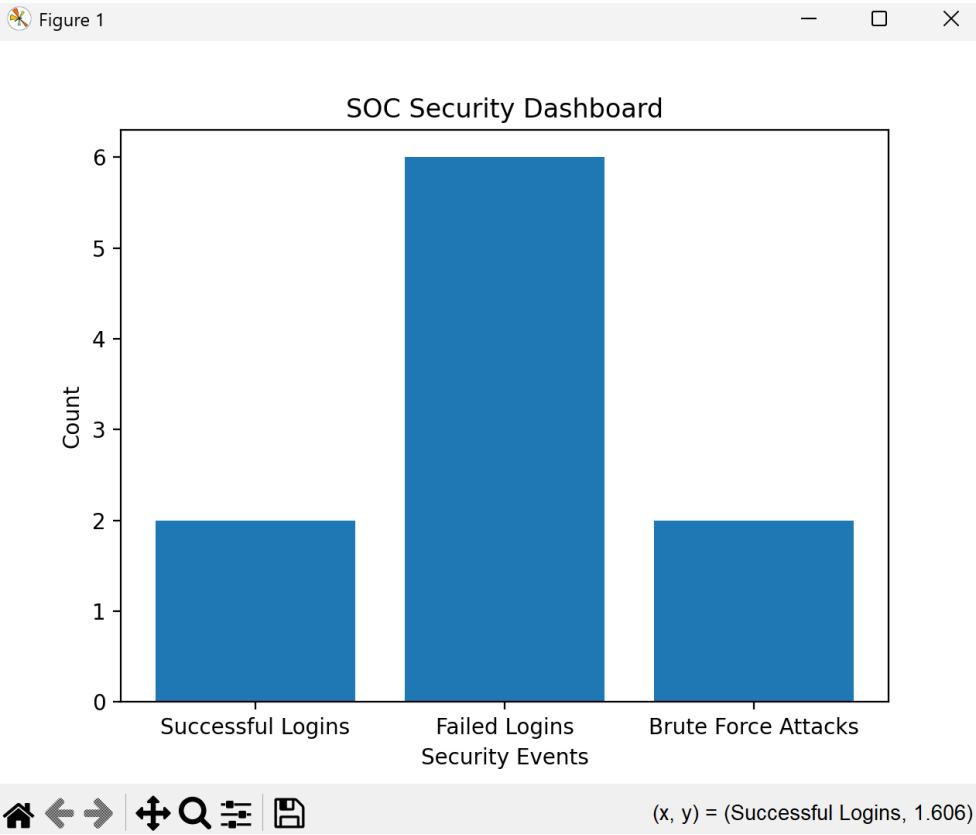
Step 3 - Write a Code

dashboard.py - C:\Users\asus\soc_project\dashboard\dashboard.py (3.14.3)

File Edit Format Run Options Window Help

```
import matplotlib.pyplot as plt

labels = ["Successful Logins", "Failed Logins", "Brute Force Attacks"]
values = [2, 6, 2]

plt.bar(labels, values)

plt.title("SOC Security Dashboard")
plt.xlabel("Security Events")
plt.ylabel("Count")

plt.show()
```

Output:



5) Make Dashboard Read Real Security Data

Step 1 - Open Dashboard Script

Open: soc_project/dashboard/dashboard.py

Delete old code.

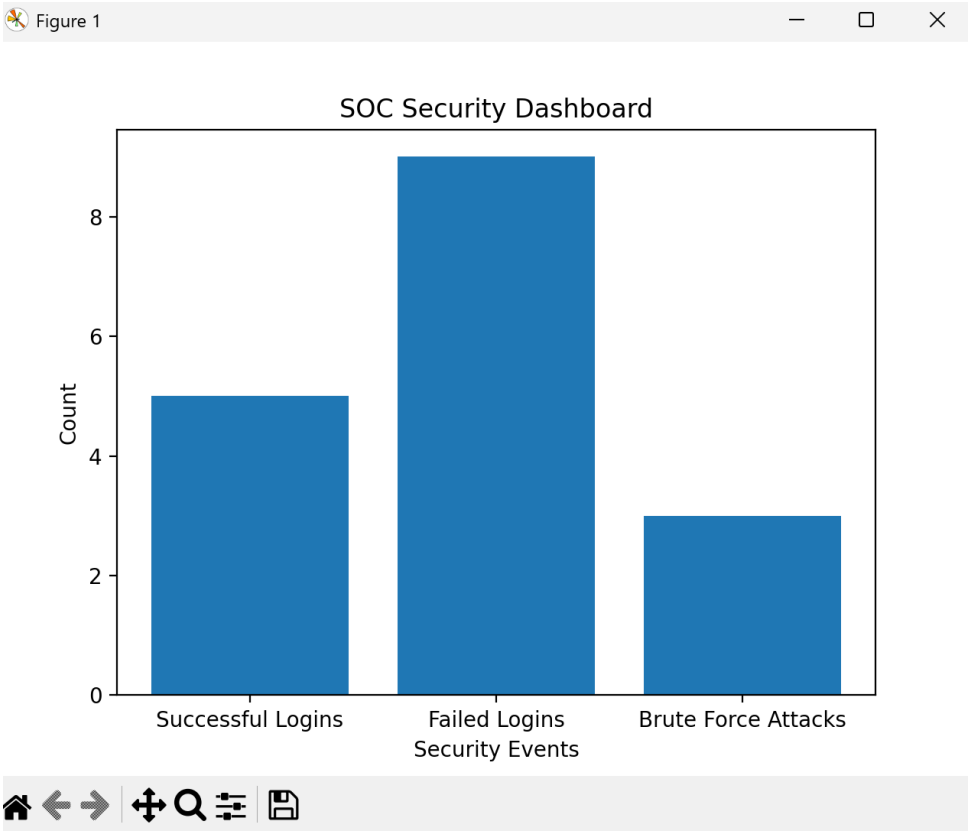
Step 2 - Write New Code for generating chart automatically

dashboard.py - C:\Users\asus\soc_project\dashboard\dashboard.py (3.14.3)

File Edit Format Run Options Window Help

```python
import matplotlib.pyplot as plt

report_file = "C:/Users/asus/soc_project/logs/security_report.txt"

success = 0
failed = 0
brute = 0

with open(report_file, "r") as file:
    lines = file.readlines()

for line in lines:

    if "Successful Logins" in line:
        success = int(line.split(":")[1])

    if "Failed Logins" in line:
        failed = int(line.split(":")[1])

    if "Brute Force Attacks" in line:
        brute = int(line.split(":")[1])

labels = ["Successful Logins", "Failed Logins", "Brute Force Attacks"]
values = [success, failed, brute]

plt.bar(labels, values)

plt.title("SOC Security Dashboard")
plt.xlabel("Security Events")
plt.ylabel("Count")

plt.show()
```

Output:



What Final Project Does

The system pipeline now works like this:

```
logs.txt  →  log_analyzer.py  →  security_report.txt  →  dashboard.py
   (logs)      (detect attacks)      (report)             (visualize)
```

6) Detect Suspicious IP Activity

We will detect **IPs that appear too many times in logs**.

This can indicate:

- scanning
- brute forcing
- automated attacks

Step 1 — Open Your Analyzer

Open:

soc_project/analyzer/log_analyzer.py

Step 2 — Add This Code (Bottom of File)

Add this after your existing detection code.

```
print("Report saved to security_report.txt")
ip_activity = {}

for log in logs:

    if "ip=" in log:

        try:
            ip = log.split("ip=")[1].strip()

            if ip in ip_activity:
                ip_activity[ip] += 1
            else:
                ip_activity[ip] = 1

        except:
            continue


print("\nSuspicious IP Activity:\n")

for ip, count in ip_activity.items():

    if count > 3:
        print("⚠ Suspicious activity detected from IP:", ip, "Events:", count)
```

Step 3 — Run Analyzer

Run:

python log_analyzer.py

Output:

```
===== SECURITY REPORT =====

Total Logs: 16
Successful Logins: 5
Failed Logins: 9
Brute Force Attacks: 3

Report saved to security_report.txt

Suspicious IP Activity:

⚠ Suspicious activity detected from IP: 192.168.1.30 Events: 4
>>>
```

Ln: 50 Col: 0

Your tool now detects:

Detection	Security Meaning
Brute Force	Many failed logins
High IP Activity	Possible bot or attacker
Log Analysis	SOC monitoring

7) Turn It into a Live SOC Monitoring Tool

Step 1 — Import Time Module

Open:

analyzer/log_analyzer.py

At the **top of the file**, add:

log_analyzer.py - C:\Users\asus\soc_p

File Edit Format Run Options W

```
import time
```

Step 2 — Create Live Monitoring Loop

Go to the **bottom of your file** and add:

```python
while True:

    print("\nMonitoring logs for new activity...\n")

    with open(log_file, "r") as file:
        logs = file.readlines()

    failed_logins = 0

    for log in logs:

        if "FAILED_LOGIN" in log:
            failed_logins += 1

    if failed_logins > 5:

        print("🚨 SECURITY ALERT 🚨")
        print("Possible brute force attack detected!")

    time.sleep(10)
```

Ln: 102 Col: 0

Step 3 — Run the Analyzer

Run:

python log_analyzer.py

Output:

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========

===== SECURITY REPORT =====

Total Logs: 32
Successful Logins: 10
Failed Logins: 18
Brute Force Attacks: 6

Report saved to security_report.txt

Suspicious IP Activity:

⚠ Suspicious activity detected from IP: 192.168.1.30 Events: 4
⚠ Suspicious activity detected from IP: 192.168.1.38 Events: 4

Monitoring logs for new activity...


Monitoring logs for new activity...


Monitoring logs for new activity...
```

Code: Change in Value failed_login = 1

```
===== SECURITY REPORT =====

Total Logs: 32
Successful Logins: 10
Failed Logins: 18
Brute Force Attacks: 6

Report saved to security_report.txt

Suspicious IP Activity:

⚠ Suspicious activity detected from IP: 192.168.1.30 Events: 4
⚠ Suspicious activity detected from IP: 192.168.1.38 Events: 4

Monitoring logs for new activity...

🚨 SECURITY ALERT 🚨
Possible brute force attack detected!

Monitoring logs for new activity...

🚨 SECURITY ALERT 🚨
Possible brute force attack detected!
```

Every **10 seconds** the system will recheck logs.

Security systems continuously:

1. Collect logs
2. Analyze them
3. Detect threats
4. Trigger alerts

8) <u>Threat Intelligence Check</u>

We will simulate checking if an IP is **malicious**.

(Using a simple method first so it stays easy.)

Step 1 — Open Your Analyzer

Open: soc_project/analyzer/log_analyzer.py

Step 2 — Create Malicious IP List

Add this **above your Suspicious IP Activity section**.

```python
# Simulated Threat Intelligence Database
malicious_ips = [
    "45.23.11.90",
    "103.45.67.89",
    "185.22.98.10"
]
```

This simulates a **threat intelligence feed**.

Step 3 — Add Threat Check Code

Below your **Suspicious IP Activity detection**, add:

```python
print("\nThreat Intelligence Check:\n")

for ip in ip_activity:

    if ip in malicious_ips:

        print("⚠ THREAT INTELLIGENCE ALERT")
        print("Malicious IP detected:", ip)
```

Step 4 — Run Your Tool

Run: python log_analyzer.py

Output:

```
========== RESTART: C:\Users\asus\soc_project\analyzer\log_analyzer.py =========

===== SECURITY REPORT =====

Total Logs: 32
Successful Logins: 10
Failed Logins: 18
Brute Force Attacks: 6

Report saved to security_report.txt

Suspicious IP Activity:

⚠ Suspicious activity detected from IP: 192.168.1.30 Events: 4
⚠ Suspicious activity detected from IP: 192.168.1.38 Events: 4

Threat Intelligence Check:

⚠ THREAT INTELLIGENCE ALERT
Malicious IP detected: 45.23.11.90

Monitoring logs for new activity...

🚨 SECURITY ALERT 🚨
Possible brute force attack detected!
```

What Just Built Is:

Project now performs **SOC-style detection pipeline**:

```
Logs
 ↓
Log Analyzer
 ↓
Attack Detection
 ↓
Suspicious IP Detection
 ↓
Threat Intelligence Lookup
 ↓
Security Alerts
 ↓
Dashboard Visualization
```

This is **very similar to real SOC workflows**.

Final Project Now Demonstrates:

✓ Log Analysis
✓ Intrusion Detection
✓ Brute Force Detection
✓ Suspicious IP Monitoring
✓ Threat Intelligence Enrichment
✓ Security Alerting
✓ Security Visualization